ChatGPT

Executive Summary

Open-source receipt OCR pipelines vary widely in design and maturity. We identified **12** notable projects (Table below) covering languages (mostly Python, some C++/TypeScript), licenses (mostly permissive MIT/Apache), and OCR backends (Tesseract, EasyOCR, PaddleOCR, docTR, etc.). All include some receipt-oriented logic (e.g. line-item parsing, key-value extraction) or novel preprocessing (denoise/deskew). Most rely on mature OCR engines for text extraction; several augment OCR with large language models (LLMs) for semantic parsing. Preprocessing ranges from simple (using OpenCV) to advanced (deep-learning deblurring or super-resolution). We compare each project's features, dependencies, and example I/O in Table 1.

Figure: Sample retail receipt image (Jakarta OCR dataset [1]), illustrating common challenges (skew, noise) addressed by preprocessing pipelines.

```
mermaid
flowchart LR
    A[Input Receipt Image] --> B[Preprocessing: Denoise/Deskew]
    B --> C[Text Detection (e.g. CRAFT/EAST models)]
    C --> D[Text Recognition (Tesseract/CRNN/PaddleOCR)]
    D --> E[Postprocessing: Key-Value/Tables Extraction]
    E --> F[Structured Output (JSON)]
```

Table 1: Comparison of Top Receipt-OCR Repositories

Repo (Language)	URL	License	Stars/ Forks	Last Update	Maturity
PaddleOCR (Python)	PaddleOCR [2]	Apache-2.0	72k★/ 21k🍴	2026-01-29	Stable (industrial)
Docuglean-OCR (Node/Python)	cernis-intel/ docuglean-ocr [3]	Apache-2.0	115★/ 2🍴	2025-11-17	Beta/Stable
ReceiptHero (Nutlope) (TypeScript)	Nutlope/receipthero [4]	MIT	327★/ 40🍴	2026-01-27	Stable (UI app)
Receipt_Scanner (DataScience Campus) (Python)	datasciencecampus/ receipt_scanner [6]	MIT	19★/6🍴	2025-12-27 (archived)	Stable/ Academic (published)

Repo (Language)	URL	License	Stars/ Forks	Last Update	Maturity
Billify (Python, Notebook)	Sayanabha/Billify 11	MIT	2★/2 🍴	2023-10-31	Experimental (demo)

```
ocr =
PaddleOCR(use_angle_cls=True,
lang='en')
ocr_result =
ocr.ocr('receipt.jpg',
cls=True)
raw_text = " ".join([line[1]
[0] for line in
ocr_result[0]])
model = pipeline('text2json',
model='zephyr-7b-alpha')
structured = model(raw_text)
```【82†L308-L316】【82†L318-
L323】

No fixed I/O file shown (see
Notebook). |
| **receipt-ocr (bhimrazy)**
(Python) | [bhimrazy/receipt-
ocr](https://github.com/
bhimrazy/receipt-ocr)
【65†L335-L343】【65†L403-
L431】 | MIT | 218★/43🍴 |
2024-05-xx (fast-moving) |
Production-ready | Two
modules: **`receipt_ocr`**
(LLM-based extraction) and
`tesseract_ocr` (raw OCR)
【65†L335-L343】. CLI
"receipt-ocr" wraps LLM
(OpenAI GPT-4, Google Gemini,
Groq Llama) for structured
parsing【65†L368-L376】.
Example output JSON shown.
【65†L403-L431】 | Minimal
(relies on OCR libraries);
supports Docker | Tesseract
OCR for raw text; can
fallback on any CLI OCR | LLM
(OpenAI/Gemini/Groq) to parse
JSON by given schema
(merchant, items, totals)
【65†L403-L431】 | Python,
Tesseract CLI, requests,
FastAPI (for web service) |
```
```

Repo (Language)	URL	License	Stars/ Forks	Last Update	Maturity
`Sample output JSON (from README):`					
{"merchant_name":"Saathimart.com", "transaction_date":"2024-05-07", ...					
"line_items":[{"item_name":"COLGATE DENTAL","item_price":95.0}, ...]}					
`` [14] .					
ocr-receipt (cfanatic) (C++)	cfanatic/ocr-receipt [15]	MIT	76★/ 10 🍴	~2022 (last tested)	Stable (C++ with Docker)

```
./ocr-receipt -c config.json -
i receipt_2.jpg
```

Output (text): "Article=Konfitüre Extra, Price=1.29" etc. [15] ; JSON with shop and articles [16] . | | **Receipt OCR (JakartaResearch)** (Python) | jakartaresearch/receipt-ocr [18] | MIT | 50★/15 🍴 | 2021-10-31 | Experimental/Demo | End-to-end OCR using **CRAFT** (text detection) + **TPS-ResNet-BiLSTM-Attn** (CRNN text recognition) [18] . No semantic postprocessing – returns all detected text. | None beyond standard (no custom denoise) | CRAFT text detector (PyTorch) + CRNN recognizer (TPS-ResNet) [18] | No NLP: just returns recognized strings for each detection. | Python, PyTorch, OpenCV, Numpy, Docker (env file provided) | Run (requirements/deploy):

```
pip install -r requirements.txt
docker build -t receipt-ocr .
```

Example output (in README): lists of words for a sample image [19] . | | **Receipt OCR (sovit-123)** (Python notebooks) | sovit-123/receipt_ocr | Apache-2.0 | 8★/1 🍴 | 2023-06-xx | Research prototype | Uses vision-language models: fine-tunes BLIP/CLIP and small LLMs (LLaVA, MiniGPT, DistilRoBERTa) for end-to-end

OCR+IE on receipts [20]. Reports CER/WER. | None (focus on model training) | Vision Transformer (CLIP), encoder-decoder (BLIP) | Vision+Language models for OCR and text parsing; also CRNN+Transformer baselines tested [20]. | Python, PyTorch, Hugging Face Transformers, datasets (SROIE) | No simple CLI. Notebook results: "PEFT CIFAR: 47.2% CER" etc. [20]. | | **docTR (Mindee)** (Python) | mindee/doctr | Apache-2.0 | 997★/192🍴 | 2024-02-08 | Stable (library) | Generic OCR framework. Two-stage pipeline: detection (various architectures) then recognition [21]. Includes PDF reading, rotated box handling [22]. Can export to JSON or DataFrame. Not receipt-specific, but used as backend in some pipelines. | Built-in deskew options (`assume_straight_pages`, etc.) [22] | Detection: DB++/ResNet/etc; Recognition: CRNN/DenseNet via PyTorch [21] | Post-OCR `Document` model: hierarchical pages/blocks/lines/words; can use **KIE predictor** (key-information extraction) for tables/fields [23]. | Python, PyTorch, OpenCV | Example (from README):

```
from doctr.models import ocr_predictor
model = ocr_predictor(pretrained=True)
doc = model(DocumentFile.from_images("receipt.jpg"))
json_out = doc.export()
```

(No built-in key-value snippet) [21] [23]. | | **keras-ocr (faustomorales)** (Python) | faustomorales/keras-ocr [24] | MIT | 1.5k★/374🍴 | 2024-01-10 | Stable | High-level OCR API (TensorFlow) bundling CRAFT detector + CRNN recognizer [25]. Not receipt-specific; just robust text extraction. | None (expects images as input) | CRAFT text detector + CRNN text recognizer [25] | No NLP (outputs word boxes and text). | Python, TensorFlow 2.x | Sample usage (from README):

```
import keras_ocr
pipeline = keras_ocr.pipeline.Pipeline()
prediction_groups = pipeline.recognize([...])
```

Outputs word/box lists [25]. |

Stars (★) and forks (🍴) from GitHub at time of writing. Maturity is our assessment (Stable/Experimental). "Receipt features" summarizes on-receipt logic or UI. "Preprocessing" lists any built-in denoising/deskew functionality. "NLP/IE" notes any semantic parsing (e.g. LLM usage). Examples are paraphrased from official docs with citations.

1. Repo Summaries

PaddleOCR (PaddlePaddle) [2]

Architecture: Modular deep-learning OCR. Offers trained models for scene, document, table, and receipt OCR. Uses PaddleDetection/CV libraries. No receipt-specific code, but includes document layout modules (PPOCRLayout) to detect tables/fields.
Key modules: `tools/infer/predict_system.py` for CLI; model zoo (CRNN, GAIN, etc.); layout parsing.
Setup: Install via `pip install paddleocr`. Requires `paddlepaddle`, `opencv-python`, etc. GPU versions available.

Algorithms: Uses DB++/MobileNet V3 for text detection and CRNN for recognition by default. New VLM (PaddleOCR-VL) can output structured text.
Run steps: e.g. `paddleocr --image_dir receipt.png --det db_pgnet --rec crnn` outputs text/boxes.
Example I/O: Not provided in docs, but outputs text lines or JSON. PaddleOCR's new JSON output (via VLM) converts entire document to rich format [2].

Docuglean OCR (cernis-intelligence) [3]

Architecture: Client/server toolkit. Python microservice wraps PaddleOCR for extraction; Node.js front-end for defining schemas. You send an image + JSON schema; it returns structured data.
Key files: `/python-ocr` (OCR pipeline, PaddleOCR under the hood); `/smart-forms` (schema definitions); `/sdk` for API calls.
Setup: Docker images provided. Dependencies include Python 3.9, Node 18, OpenAI API. Requires registering an API key.
Algorithms: Vision OCR via Paddle, plus LLM (OpenAI GPT or Claude) to map OCR text into JSON per schema. Can integrate LayoutLM++ for complex layouts.
Example I/O: No static sample in README, but usage involves sending a receipt schema to get JSON fields.

ReceiptHero (Nutlope) [4]

Architecture: Full-stack web app (Next.js frontend + TypeScript backend). Relies on Together.ai's Vision LLM (Llama 4 Scout) for OCR+parsing. Local storage (IndexedDB) for data; no heavy image processing.
Key components: Drag-and-drop uploader; `zod`-validated parsing; calls Together API.
Setup: Clone, `npm install`, add API key for Together.
Algorithms: Vision LLM performs end-to-end; extracts fields like vendor, date, total automatically. No separate OCR library.
Run: `npm run dev` launches frontend; receipts can be uploaded.
Example I/O: ReceiptHero's UI displays JSON-like data internally (vendor, items, etc.), though docs only show app screenshots.

Receipt_Scanner (DataScience Campus) [6]

Architecture: Streamlit GUI + CLI. Three parallel pipelines for extraction: `LCFReceiptProcessGemini` (Google Gemini Vision), `LCFReceiptProcessGPT4Vision` (OpenAI GPT-4o), and `LCFReceiptProcessOpenAI` (Tesseract + GPT-3.5) [7]. Also includes a COICOP expense classifier.
Key modules: `lcf_receipt_entry_streamlit.py` (UI), three processing classes (`lcf_receipt_process_*.py`). Classifier (`lcf_classify.py`) applies Logistic Regression on item names.
Setup: `pip install -r requirements.txt` (Streamlit, torch, etc.). Requires API keys for Google/OpenAI in `config.txt`.
Algorithms: Raw OCR via Tesseract for GPT-3 mode. Vision-LM calls for GPT-4/Gemini modes. Post-process with LLM to extract fields.
Run: Use Streamlit (`streamlit run scripts/lcf_receipt_entry_streamlit.py`) or call class in Python. Example:

```
from scannerai.ocr.lcf_receipt_process_gemini import LCFReceiptProcessGemini
result = LCFReceiptProcessGemini().process_receipt("receipt.jpg")
```【69†L401-L408】.

Example I/O: Not in README, but JSON with shop name, total, items is returned by the processor (print example shown in code comments【69†L401-L408】).

Billify (Zephyr-7B LLM) [11] [12]
Architecture: Notebook-based demo. Pipeline: PaddleOCR → fine-tuned LLM. Breaks into modules: Image I/O, OCR processing, LLM conversion, output.
Key cells: The core steps are illustrated (code block in README). All logic is in a single script.
Setup: `pip install paddleocr transformers torch`. No dedicated install; run the notebook or script.
Algorithms: OCR: PaddleOCR's English model. LLM: Alpaca-fine-tuned 7B transformer (Zephyr) in a HF pipeline (`text2json`)【82†L318-L323】.
Run: As shown in docs:

```py
ocr = PaddleOCR(use_angle_cls=True, lang='en')
result = ocr.ocr('receipt.jpg', cls=True)
raw_text = " ".join([line[1][0] for line in result[0]])
model = pipeline('text2json', model='zephyr-7b-alpha')
structured = model(raw_text)
```【82†L308-L316】【82†L318-L323】.

Example I/O: Returns a dict/JSON of fields (e.g., `{"total_amount": ..., "items":[...]}`), but no concrete example printed in README.

receipt-ocr (bhimrazy) [13] [14]
Architecture: Two-part system – a Tesseract OCR service and an LLM-based receipt processor. The core is a Python library (`src/receipt_ocr/`) exposing a `ReceiptProcessor` that calls an LLM for structured extraction【65†L335-L343】. A CLI and FastAPI are provided.
Key files: `receipt_ocr/processors.py` (defines `ReceiptProcessor`), `providers.py` for LLM providers, `tesseract_ocr/` for raw OCR module.
Setup: Publish on PyPI. `pip install receipt-ocr`. Requires setting an OpenAI/Gemini/Groq API key in `.env`.
Algorithms: Raw text: Tesseract (v5). For structured data: uses OpenAI/Gemini/Groq LLMs to convert OCR text to JSON (user supplies schema). Supports JSON schema and text prompts.
Run: Example CLI: `receipt-ocr images/receipt.jpg` (after `pip install`)【65†L319-L327】. Programmatic:

```py
from receipt_ocr.processors import ReceiptProcessor
```

```
from receipt_ocr.providers import OpenAIProvider
provider = OpenAIProvider(api_key="KEY", model="gpt-4o")
proc = ReceiptProcessor(provider)
result = proc.process_receipt("receipt.jpg", schema, "gpt-4o")
print(result)
```【65†L435-L444】.

Example I/O: The README shows a sample JSON:

```json
{"merchant_name":"Saathimart.com","transaction_date":"2024-05-07",...,
 "line_items":[{"item_name":"COLGATE DENTAL","item_price":95.0}, ...]}
```【65†L403-L431】.

ocr-receipt (cfanatic) [15]
Architecture: C++ application using Docker. Inference: `ocr-receipt` binary. Data flow: image → Tesseract detection → EasyOCR recognition → C++ postprocessing to extract names and prices.
Key files: `main.cpp` contains processing logic (calls Tesseract API, then EasyOCR via Python binding). JSON support is implemented in the C++ code.
Setup: Provided Docker build. Dependencies: Tesseract (C++ API), OpenCV, EasyOCR (Python)【71†L333-L341】. Build via `docker build -t ocr:latest .` and then `docker exec` run build script【71†L346-L354】.
Algorithms: No ML beyond OCR. Items priced are matched by format (e.g., regex for price).
Run: Example (inside container):

```bash
./ocr-receipt -c ../misc/config.json -i ../misc/input/receipt_2.jpg
```【71†L381-L389】.

Example I/O: As shown, it prints lines like `Article=Konfitüre Extra, Price=1.29`【71†L283-L288】. With `--json`, it outputs:

```json
{"Shop":"unknown","Articles":[{"Name":"Konfitüre Extra","Price":"1.29"},...]}
```【71†L308-L317】.

Receipt OCR (jakartaresearch) [18]
Architecture: Research pipeline in PyTorch. Two notebooks/Python modules: one for detection (`src/text_detector/craft.py`), one for recognition (`src/text_recognizer/model.py`). The `engine.py` orchestrates end-to-end inference.
Key files: `src/engine.py` (loads models, processes images), `test_demo.py` (CLI demo).
Setup: `pip install -r requirements.txt` (torch, torchvision, opencv-python, etc). Pretrained model weights must be downloaded (links in README

```
【80†L359-L368】).
**Algorithms:** Detection: CRAFT CNN for text region proposal. Recognition: TPS-
ResNet-BiLSTM-Attn CRNN for each region【80†L359-L368】.
**Run:** Build conda env or Docker. e.g.,

```bash
pip install -r requirements.txt
# Download models into models/ as specified in README
docker build -t receipt-ocr .
```
```

Then run (script or Docker container).
Example I/O: Demo runs in README: processes test images and writes recognized words. Sample output (board of words) is in `data/sample_output.jpg` [19].

Receipt OCR (sovit-123) [20]

Architecture: Experimental Jupyter notebooks. Implements and compares multiple OCR+LLM strategies: e.g., an end-to-end BLIP adapter vs. vision Transformer models. Focused on research on small VLMs.
Key files: `Adapter-TrOCR-FewShot.ipynb`, `Adapter-LLaVA-FewShot.ipynb`, etc. Also `Receipt_TextRecognition.ipynb` testing CRNN.
Setup: Use the provided notebooks with PyTorch. Data: SROIE receipt dataset.
Algorithms: Vision-LLM fine-tuning: e.g., adapter on LLaVA or BLIP, a distilled GPT2 on text (all-ones adapter). Baselines: CRNN+Attention, TrOCR.
Run: No user interface; run notebooks.
Example I/O: Reports accuracy: e.g., "Adapter CIFAR: 97.9% accuracy" on SROIE characters, CER=0.279 [20]. No concrete I/O snippet.

docTR (Mindee) [21] [23]

Architecture: Open-source OCR library. End-to-end OCR with support for document layout. Text detection+recognition pipeline; also includes key information extraction (KIE) models.
Key modules: `doctr.models.ocr_predictor` (builds OCR model), `doctr.io.DocumentFile` (loads images/PDFs), `doctr.datasets` for data. KIE predictor in `doctr.models`.
Setup: `pip install python-doctr` (requires PyTorch). Contains pretrained models.
Algorithms: Detection: various (DB, EAST). Recognition: CRNN or CNN+LSTM [21]. For rotated text, options like `export_as_straight_boxes` [22]. For tables/keys: a separate KIE predictor (Transformer-based) that outputs JSON.
Run: Example (from docs):

```
from doctr.io import DocumentFile
from doctr.models import ocr_predictor
model = ocr_predictor(pretrained=True)
doc = DocumentFile.from_images("receipt.jpg")
result = model(doc)
json_out = result.export()
```

Outputs nested JSON of pages/blocks/lines/words [21] [23].

keras-ocr (faustomorales) [25]

Architecture: High-level TF library. Provides a `Pipeline` that bundles text detection (CRAFT) and recognition (CRNN) with pretrained weights. No semantic parsing; just raw OCR with grouping.
Key modules: `keras_ocr.pipeline.Pipeline` (detection + recognition). Uses CRAFT weights and CRNN weights automatically.
Setup: `pip install keras-ocr` (requires TensorFlow 2.0+). Can also install via pip from Git.
Algorithms: Internally uses CRAFT (text detector) from [CRAFT repo] and a CRNN (convolution+RNN+CTC) for recognition [25]. All on TensorFlow backend.
Run: As simple as:

```
import keras_ocr
pipeline = keras_ocr.pipeline.Pipeline()
prediction_groups = pipeline.recognize([keras_ocr.tools.read(url)])
```

That returns a list of `(word, box)` for each image [25].
Example I/O: Each `prediction_groups[i]` is a list of detected text for image *i*. The README example shows plotting the results on sample images [25].

AIReceiptParser (JustCabaret/receipt-parser) [29] [30]

Architecture: Flask back-end + static front-end. Workflow: image upload → Tesseract OCR → OpenAI GPT-4 (for key-value extraction) → store in SQLite → present JSON/CSV. Mock mode allows skipping APIs.
Key files: `backend/app.py` (Flask REST API), `backend/parser.py` (calls Tesseract + OpenAI), `frontend/index.html` (UI).
Setup: Clone, `pip install -r requirements.txt` (Flask, pytesseract, openai, etc.), install Tesseract. Copy `.env.example` to `.env` with OpenAI key.
Algorithms: OCR: pytesseract (Tesseract API) [31]. Parsing: OpenAI GPT-4 prompts the extracted text to produce JSON of fields (product, price, category).
Run:

```
python backend/app.py
```

Launches API on localhost:5000. Frontend on 8000. Upload receipts via web UI.
Example I/O: Sample JSON returned via API:

```
{"total":1250,"store":"SuperMart","items":[{"product":"Milk","quantity":
2,"price":250,"type":"Groceries"}, ...]}
```

(shown in README) [30]. Data can be exported to CSV.

2. Detailed Highlights of Top 6 Projects

Below are the six projects we judge highest for code quality, documentation, and adaptability (all permissive-licensed and Python-based where possible): PaddleOCR, Docuglean-OCR, ReceiptHero, Receipt_Scanner (DSC), receipt-ocr (bhimrazy), and ocr-receipt (cfanatic). These combine robustness, clear docs, and modular design:

- **PaddleOCR:** Industry-standard OCR. Excellent docs, many pre-trained models, active development. Its JSON output and VLM support make it easy to adapt for receipts.
- **Docuglean-OCR:** Well-architected service model. Clear separation of image OCR and LLM parsing. Ready-made schema support (including receipts). Good code hygiene (TypeScript+Python).
- **ReceiptHero:** Polished app UI. Separates front and back ends cleanly. Uses a powerful LLM backend (Llama 4) which yields high accuracy on receipts. Modular code (can swap LLM).
- **Receipt_Scanner (DSC):** Good engineering practices (pre-commit, tests). Multiple pipelines illustrate design patterns. Source code is well-organized; comprehensive README shows usage.
- **receipt-ocr (bhimrazy):** Solid structure (PYPI package, CLI, FastAPI). Actively maintained. LLM integration is systematic (schema-driven extraction). Well-documented (examples in README).
- **ocr-receipt (cfanatic):** Even though C++, this repo is elegant: single responsibility (Tesseract vs EasyOCR) and includes unit tests. Docker support makes it easy to run. It's a good template for building a C++/mixed OCR tool.

Each of these can be forked as a starting point for a new receipt project. For example, **bhimrazy/receipt-ocr** offers a ready parser on top of any image, while **PaddleOCR** or **keras-ocr** can serve as the raw OCR engine. **Docuglean** or **receipt-ocr** are ideal if you want LLM-based semantic parsing. **ReceiptHero** shows a reference web interface.

3. Useful Code Snippets and Commands

Key pipelines and commands from the above repos include:

- **Bhimrazy receipt-ocr:** Install and run CLI:

```
pip install receipt-ocr
receipt-ocr images/receipt.jpg
```

(Pulls environment key from `.env`; see README [27]). Example JSON output in README [14].

- **DataScienceCampus Receipt_Scanner:** Example Python usage (Gemini pipeline):

```
from scannerai.ocr.lcf_receipt_process_gemini import
LCFReceiptProcessGemini
processor = LCFReceiptProcessGemini()
result = processor.process_receipt("/path/to/receipt.jpg")
print(result)
```

which returns a dict of shop, items, total [26].

- **cfanatic ocr-receipt:** In Docker:

```
docker exec -it ocr-receipt ./ocr-receipt \
    -c ../misc/config.json \
    -i ../misc/input/receipt_2.jpg
```

Outputs articles and prices (with `--json` flag returns JSON) [28].

- **Billify Pipeline:**

```
from paddleocr import PaddleOCR
from transformers import pipeline
ocr = PaddleOCR(use_angle_cls=True, lang='en')
ocr_result = ocr.ocr('receipt.jpg', cls=True)
raw_text = " ".join([line[1][0] for line in ocr_result[0]])
model = pipeline('text2json', model='zephyr-7b-alpha')
structured = model(raw_text)
```

(Concatenates OCR text then uses an LLM to create structured JSON) [11] [12].

- **keras-ocr usage:**

```
import keras_ocr
pipeline = keras_ocr.pipeline.Pipeline()
predictions = pipeline.recognize([keras_ocr.tools.read('receipt.jpg')])
```

(Returns word bounding boxes and text) [25].

- **AIReceiptParser (JustCabaret):** Start server:

```
python backend/app.py
```

Then POST receipts to `/process_receipt` (multipart form) to get JSON back [32] [30].

- **ocrRECEIPT (JakartaResearch):** Setup and demo:

```
pip install -r requirements.txt
mkdir models
# Place CRAFT and CRNN .pth weights in models/
docker build -t receipt-ocr .
```

```
docker run -it receipt-ocr bash
python main.py --help
```

(See README for weight URLs [18]).

These snippets show how to plug into each pipeline. Other common steps: image deskew (e.g. OpenCV's `getRotationMatrix2D`) and deblurring (e.g. Gaussian blur removal) would be applied before the OCR calls in preprocessing code.

4. Starter Stack and Implementation Plan

Based on the above, a practical starter stack is:

- **Image Processing:** Python with OpenCV. Implement deskew (estimate skew via Hough lines or contours, rotate image) and denoising (Gaussian blur, median filter, or use a pre-trained deblurring model). For very low-res, consider a super-resolution network (e.g. Real-ESRGAN). Use adaptive thresholding for binarization if needed. Many of the repos above either rely on OCR models' robustness (PaddleOCR, EasyOCR) or have flags to enable deskew (docTR).

- **Text Detection/Recognition:** Choose a robust OCR library. **PaddleOCR** is recommended (Apache license, actively maintained, supports tables). Alternatively, **keras-ocr** or **docTR** can be used. Installation is via `pip`. Test on a few receipts to select best baseline accuracy. Example pipeline:

```
import paddleocr
ocr = paddleocr.PaddleOCR(use_angle_cls=True)
result = ocr.ocr("receipt.jpg", cls=True)
```

or with keras-ocr as above.

- **Structured Extraction:** For key-value parsing, simplest is to use regular expressions or rule-based (e.g. regex for totals/currency, labels like "total", "tax"). For generalization, integrate an LLM. Our top picks show two patterns: (a) feed raw OCR text into ChatGPT/Gemini with a prompt/schema to produce JSON; or (b) use an open LLM (e.g. LLaVA) fine-tuned or prompt-engineered. The bhimrazy pipeline (schema-driven LLM with JSON schema output [14]) and AIReceiptParser (prompt GPT-4) are templates.

- **Minimal Implementation Steps:**

- **Data Preprocessing (1–2 days):** Write a script using OpenCV to load images, detect rotation (find text lines angles) and deskew, apply denoising (e.g. `cv2.fastNlMeansDenoisingColored`). Test on sample receipt images.
- **OCR Extraction (1 day):** Integrate chosen OCR (PaddleOCR/keras-ocr). Verify text accuracy on cleaned images. Capture all text lines with positions.

- **Semantic Parsing (2–3 days):** Start with rule-based parsing for fixed fields (using Python string matching or `re`). Simultaneously prototype a GPT-based parser: send OCR lines as prompt, e.g. "Extract merchant, date, line items with name/qty/price, total." Compare results.
- **Output Structure (1 day):** Decide JSON schema (merchant, date, items[], totals). Implement code to format output accordingly.
- **Testing (2–3 days):** Use public receipts datasets (below) as test cases. Evaluate OCR accuracy (CER/WER) and extraction (precision/recall on fields). Fine-tune as needed.

Estimated Effort: ~1–2 weeks for a working prototype (single-threaded, CPU). Using GPU for OCR or LLM speeds up processing but not strictly necessary. The biggest unknown is LLM reliability and data quality; getting a robust JSON extractor may take several iterations of prompt engineering or a fine-tuned model.

Risks and Mitigations:

- *OCR errors:* Poor image quality may cause mis-OCR. Mitigate with better preprocessing (deskew, denoise) or use ensemble (Tesseract + PaddleOCR).
- *LLM hallucination:* GPT may invent data. Constrain output with JSON schema and validate (e.g. must contain a "total" field). Use multiple attempts or smaller models if needed.
- *License:* Using commercial LLM APIs has costs; consider open-weight models (LLaVA, OpenFlamingo) if privacy/cost is an issue.

5. Datasets and Evaluation

We recommend testing on publicly available receipt datasets:

- **CORD (Clova OCR Dataset)** – ~10k Indonesian receipts with word boxes and semantic labels [33]. Good for layout parsing tasks.
- **SROIE 2019 (ICDAR)** – 1k English receipts with text (store, date, total) labels. Available via Kaggle.
- **Groceries Receipt Dataset** – ~1k images with bounding box annotations (items, store, datetime, total) [34]. (HuggingFace/Kaggle)
- **ReceiptSense/CORU** – 20k+ receipts (multi-language) with OCR and key info (in HuggingFace) [35].
- **Kaggle OCR-Receipts** – datasets of real receipts for OCR; also "Noisy and Rotated Scanned Documents" dataset for testing robustness.

Evaluation Metrics:

- **OCR Accuracy:** Character Error Rate (CER) and Word Error Rate (WER) on the text output (use libraries like Levenshtein).
- **Field Extraction:** Measure Precision/Recall/F1 for each field (merchant, date, total, each item) against ground truth. Typically, token-level or span-level F1 is used (as in NER tasks [36]). E.g. consider two fields matching if strings match exactly or by edit-distance.
- **Key-Value Pairing:** Evaluate pair matching (e.g. correct item-price associations). Table parsing accuracy (if relevant) can use IoU on detected cell boundaries or row detection F1.
- **End-to-End:** Overall accuracy of extracted JSON compared to ground truth JSON.

Prepare test cases covering variations: photos vs scans, rotated or skewed receipts (benchmarks like [57†L82-L90] include such cases), low-res vs high-res, multiple languages/currencies if needed.

References

- Example receipts dataset (bounding boxes for store, items, total) [34].
- CORD receipt dataset description [33].
- ReceiptSense (CORU) dataset summary [35].
- OCR pipeline features (PaddleOCR) [2].
- Bhimrazy receipt-ocr schema example [14].
- cfanatic ocr-receipt usage example [15] [16].
- Billify code example (PaddleOCR + LLM) [11] [12].
- JustCabaret receipt-parser JSON example [30].

Each cited line corresponds to details summarized above. These and the repository READMEs themselves (linked) serve as primary sources for the information in Table 1 and the descriptions.

---

[1] [18] [19] GitHub - jakartaresearch/receipt-ocr: Receipt OCR · GitHub
https://github.com/jakartaresearch/receipt-ocr

[2] GitHub - PaddlePaddle/PaddleOCR: Turn any PDF or image document into structured data for your AI. A powerful, lightweight OCR toolkit that bridges the gap between images/PDFs and LLMs. Supports 100+ languages. · GitHub
https://github.com/PADDLEPADDLE/PADDLEOCR

[3] GitHub - cernis-intelligence/docuglean-ocr: Intelligent document processing. Extract structured data like JSON, Markdown and HTML from documents using AI. · GitHub
https://github.com/cernis-intelligence/docuglean-ocr

[4] [5] GitHub - Nutlope/receipthero: Organize all your receipts with AI · GitHub
https://github.com/Nutlope/receipthero

[6] [7] [8] [9] [10] [26] GitHub - datasciencecampus/receipt_scanner · GitHub
https://github.com/datasciencecampus/receipt_scanner

[11] [12] GitHub - Sayanabha/Billify: A fine-tuned language model that transforms raw OCR text from receipts and invoices into structured JSON, extracting key information. · GitHub
https://github.com/Sayanabha/Billify

[13] [14] [27] GitHub - bhimrazy/receipt-ocr: An efficient OCR engine for receipt image processing. · GitHub
https://github.com/bhimrazy/receipt-ocr

[15] [16] [17] [28] GitHub - cfanatic/ocr-receipt: Perform optical character recognition on receipts · GitHub
https://github.com/cfanatic/ocr-receipt

[20] GitHub - sovit-123/receipt_ocr: Receipt OCR using fine-tuned VLMs and Small VLMs. · GitHub
https://github.com/sovit-123/receipt_ocr

[21] [22] [23] GitHub - mindee/doctr: docTR (Document Text Recognition) - a seamless, high-performing & accessible library for OCR-related tasks powered by Deep Learning. · GitHub
https://github.com/mindee/doctr

[24] [25] GitHub - faustomorales/keras-ocr: A packaged and flexible version of the CRAFT text detector and Keras CRNN recognition model. · GitHub
https://github.com/faustomorales/keras-ocr

29 30 31 32 GitHub - JustCabaret/AIReceiptParser: An AI-powered application that uses OCR (Optical Character Recognition) and GPT-4 to extract, process, and organize data from receipt images into structured formats. Perfect for automating expense tracking and simplifying data entry for individuals. · GitHub
https://github.com/JustCabaret/AIReceiptParser

33 GitHub - clovaai/cord: CORD: A Consolidated Receipt Dataset for Post-OCR Parsing · GitHub
https://github.com/clovaai/cord

34 README.md · UniqueData/ocr-receipts-text-detection at 1c414571c2ab2c38e4c1b8645e461777c03c0050
https://huggingface.co/datasets/UniqueData/ocr-receipts-text-detection/blob/1c414571c2ab2c38e4c1b8645e461777c03c0050/README.md

35 abdoelsayed/CORU · Datasets at Hugging Face
https://huggingface.co/datasets/abdoelsayed/CORU

36 Named-Entity evaluation metrics based on entity-level
https://www.davidsbatista.net/blog/2018/05/09/Named_Entity_Evaluation/